



杭州电子科技大学
《编译原理课程实践》
实验报告

题 目：_____词法分析_____

学 院：_____计算机学院_____

专 业：_____计算机科学与技术_____

班 级：_____22052312_____

学 号：_____22050201_____

姓 名：_____黄江晔_____

完成日期：_____2024. 12. 11_____

一、实验目的

程序要能够查出 SysY 源代码中可能包含的词法错误：

词法错误(错误类型代码为 A)：出现 SysY 词法中未定义的字符以及任何不符合 SysY 词法单元定义的字符。

程序在输出错误提示信息时，需要输出具体的错误类型、出错的位置（源程序的行号）以及相关的说明文字。

二、实验内容与实验要求

程序输入是一个包含 SysY 源代码的文本文件，程序需要能够接受一个输入文件名作为参数。例如：假设程序名为 ss，输入文件名为 test1.sy，程序和输入文件都在当前目录下，那么在 Linux 命令行下运行 ./ss test1.sy 即可获得 test1.sy 作为输入文件的输出结果。

实验一要求通过标准输出打印程序的运行结果（也可以单独输出到文件，格式保持一致）。对于那些包含词法错误的输入文件，只要输出相关的词法错误信息即可（如果你能有出错恢复策略，把所有可能的错误找出来，更好）。要求输出的错误信息包括错误类型、出错的行号以及说明文字，格式如下：

Error type [错误类型] at line [行号] : [说明文字]

说明文字的内容不做具体要求，但是错误类型和行号要正确。假设输入文件中可能包含一个或者多个错误，但同一行最多只有一个错误。每一条错误信息在输出中单独占一行。

对于没有任何词法错误的输入文件，程序要将识别到的词法单元列表按顺序输出打印，每行代表一个词法单元的元组：〈词法单元类型，属性值〉。词法单元类型编码方式暂时不做统一要求，可以先自己设计。

三、设计方案与算法描述

1、头文件的编写

“translator.h”头文件内对需要识别的词法类型 (TYPE, OCTAL_CONST, ...) 进行了 enum 枚举，因为 enum 枚举默认从 0 开始，而 yylex() 读取完文件也返回 0 会造成误解，因此将枚举的第一项类型赋值为 1。

```
#include <stdio.h>
enum target
{
    TYPE = 1,
    KEYWORD,
    OPERATOR,
    COMPARISON,
    NONZERO,
    DIGIT
```

2、Lexical.l 文件的编写、正则表达式的设计

使用 flex 进行 SysY 语言的词法分析需要提交给 Flex 一个 .l 文件来告诉它词法规则。

Lexical.l 文件的各部分如下：

第一部分，使用`%option yylineno` 开启 flex 自带的辅助功能，提供行号记录变量 `yylineno`，从而报错信息中可以定位行号。

```
GNU nano 6.2
%option yylineno
```

第二部分，引用 `translator.h` 头文件。

```
%{
    #include "translator.h"
%}
```

第三部分，是定义段，采用正则表达式匹配规则来识别不同的 Token 类型。

- (1) 类型：TYPE 匹配 `int` 或 `void`
- (2) 关键字：KEYWORD 匹配 `if`, `else`, `while`, `break`, `continue`, `return`
- (3) 运算符：OPERATOR 匹配 `+`, `-`, `*`, `/`, `%`, `=`, `!`
- (4) 比较符：COMPARISON 匹配 `==`, `!=`, `>`, `<`, `>=`, `<=`
- (5) 常量：
 - 十进制常量：DEC_CONST 匹配 `[1-9][0-9]*`
 - 八进制常量：OCTAL_CONST 匹配 `0[0-7]+`
 - 十六进制常量：HEX_CONST 匹配 `0x[0-9a-fA-F]+`
- (6) 标识符：ID 匹配字母或下划线开始，后跟字母、数字或下划线
- (7) 分隔符和注释：通过匹配符号 `(, , {, }, [, ], ;, ,, /* ... */`, `// ...`

```
TYPE          int|void|float
KEYWORD       if|else|while|break|continue|return
OPERATOR      "+"|"-"|"!"|"*"|"%"|"/"|"="
COMPARISON    "=="|"!="|">"|"<"|">="|"<="
NONZERO       [1-9]
DIGIT         [0-9]
LETTER        [A-Za-z]
OCTAL_DIGIT   [0-7]
OCTAL_CONST   0{OCTAL_DIGIT}*
ILLEGAL_OCTAL_CONST 0[0-9a-wy-zA-WY-Z]({LETTER}|{DIGIT})*
HEX_PREFIX    0x|0X
HEX_DIGIT     [0-9a-fA-F]
HEX_CONST     {HEX_PREFIX}{HEX_DIGIT}+
ILLEGAL_HEX_CONST {HEX_PREFIX}({LETTER}|{DIGIT})*
NONDIGIT      {LETTER}|"_"
ID            {NONDIGIT}({DIGIT}|{NONDIGIT})*
DEC_CONST     {NONZERO}{DIGIT}*
FLOAT_CONST   [0-9]+"."[0-9]+([eE][+-]?[0-9]+)?
COMMENT1      "/*"[^*]*"*/"
COMMENT2      "//".*
```

第四部分，是规则段，定义需要被识别出来的词法类型(可以用第三部分中的定义来组合)，以及识别到该类型时需要进行的操作。每一次 Flex 都会使用尽可能长的字符串进行匹配，即选择最长字符串可以匹配到的类型。但当同一个长度的字符串匹配了多个类型时，默认选择第一个匹配到的类型。

```

%%

{TYPE}                { printf("\033[1;32mTYPE\033[0m\t\t%s\n",yytext); return TYPE; }
{OCTAL_CONST}         { printf("\033[1;32mOCTAL_CONST\033[0m\t\t"); return OCTAL_CONST; }
{ILLEGAL_OCTAL_CONST} { return ILLEGAL_OCTAL_CONST; }
{HEX_CONST}           { printf("\033[1;32mHEX_CONST\033[0m\t\t"); return HEX_CONST; }
{ILLEGAL_HEX_CONST}   { return ILLEGAL_HEX_CONST; }
{DEC_CONST}           { printf("\033[1;32mDEC_CONST\033[0m\t\t%s\n",yytext); return DEC_CONST; }
{FLOAT_CONST}         { printf("\033[1;32mFLOAT_CONST\033[0m\t\t%s\n",yytext); return FLOAT_CONST; }
{KEYWORD}             { printf("\033[1;32mKEYWORD\033[0m\t\t%s\n",yytext); return KEYWORD; }
{ID}                  { printf("\033[1;32mID\033[0m\t\t%s\n",yytext); return ID; }
{OPERATOR}            { printf("\033[1;32mOPERATOR\033[0m\t\t%s\n",yytext); return OPERATOR; }
{COMPARISON}          { printf("\033[1;32mCOMPARISON\033[0m\t\t%s\n",yytext); return COMPARISON; }
"("                  { printf("\033[1;32mLPARENT\033[0m\t\t%s\n",yytext); return LPARENT; }
")"                  { printf("\033[1;32mRPARENT\033[0m\t\t%s\n",yytext); return RPARENT; }
"["                  { printf("\033[1;32mLBRACKET\033[0m\t\t%s\n",yytext); return LBRACKET; }
"]"                  { printf("\033[1;32mRBRACKET\033[0m\t\t%s\n",yytext); return RBRACKET; }
"{"                  { printf("\033[1;32mLBRACE\033[0m\t\t%s\n",yytext); return LBRACE; }
"}"                  { printf("\033[1;32mRBRACE\033[0m\t\t%s\n",yytext); return RBRACE; }
";"                  { printf("\033[1;32mSEMICN\033[0m\t\t%s\n",yytext); return SEMICN; }
","                  { printf("\033[1;32mCOMMA\033[0m\t\t%s\n",yytext); return COMMA; }
"&&"                { printf("\033[1;32mAND\033[0m\t\t%s\n",yytext); return AND; }
"||"                 { printf("\033[1;32mOR\033[0m\t\t%s\n",yytext); return OR; }
{COMMENT1}|{COMMENT2} { }
[ \t\n]              { }
.                     { return UNEXPECTED; }

%%

```

第五部分，是自定义代码，会直接复制到生成代码的最后。前面词法识别规则会直接被 Flex 翻译为一个 `yylex()` 函数，该函数即为匹配词法类型的函数。另外，Flex 还提供了两个全局变量 `yytext` 和 `yylen`，分别表示刚刚匹配到类型的字符串与该字符串的长度。

`yylex()` 是 Flex 自动生成的扫描函数，它从输入中逐字符扫描，匹配并返回识别到的 Token 类型，当 `yylex()` 返回 0 时，表示已达到输入的末尾，循环结束。根据 `yylex()` 返回的 `token_type`，程序进行不同的处理。当扫描到一个无法识别的字符时，返回 `UNEXPECTED` 类型，输出错误信息，表明出现了无效字符，并打印出当前的行号 `yylineno` 和字符 `yytext`，错误类型 A，表示一个无效的字符。

当识别到一个八进制常量（如 01234）时，返回 `OCTAL_CONST` 类型，程序通过循环遍历常量的每一位，计算其对应的十进制值，跳过首位 0（八进制常量以 0 开头），使用公式 $sum = sum * 8 + (yytext[i] - '0')$ 累加每一位数字的值，最后输出转换后的十进制数值。

当识别到一个十六进制常量（如 0x1A 或 0Xabc）时，返回 `HEX_CONST` 类型，程序跳过前缀 0x 或 0X，从第三个字符开始处理十六进制数，根据字符的值（数字 0-9 或字母 a-f, A-F），计算其对应的十进制值：数字字符（0-9）直接转换，字母字符（a-f 或 A-F）则通过 `switch` 语句进行转换，a/A 对应 10，b/B 对应 11，依此类推，最后输出转换后的十进制值。

如果遇到浮点数常量，返回 `FLOAT_CONST` 类型。

如果遇到一个非法的八进制常量（如 089 或包含不合法字符的八进制数），返回 `ILLEGAL_OCTAL_CONST` 类型，输出错误信息，表明这是一个非法的八进制常量，并显示该常量。

如果遇到一个非法的十六进制常量（如 0xgh 或包含不合法字符的十六进制数），返回 `ILLEGAL_HEX_CONST` 类型。输出错误信息，表明这是一个非法的十六进制常量，并显示该常量。

```

int main()
{
    int token_type;
    while (token_type = yylex())
    {
        if (token_type == UNEXPECTED)
            printf("\033[1;31mError type A at Line %d: Invalid character \"%s\"\033[0m\n", yylineno, yytext);
        else if (token_type == OCTAL_CONST)
        {
            int sum = 0;
            for (int i = 1; i < yyleng; ++i)
                sum = sum * 8 + (yytext[i] - '0');
            printf("%d\n", sum);
        }
        else if (token_type == HEX_CONST)
        {
            int sum = 0;
            for (int i = 2; i < yyleng; ++i)
            {
                if (yytext[i] <= '9' && yytext[i] >= '0')
                    sum = sum * 16 + (yytext[i] - '0');
                else
                {
                    switch (yytext[i])
                    {
                        case 'a':
                        case 'A':
                            sum = sum * 16 + 10;
                            break;
                        case 'b':
                        case 'B':
                            sum = sum * 16 + 11;
                            break;
                        case 'c':
                        case 'C':
                            sum = sum * 16 + 12;
                            break;
                        case 'd':
                        case 'D':

```

```

                        case 'D':
                            sum = sum * 16 + 13;
                            break;
                        case 'e':
                        case 'E':
                            sum = sum * 16 + 14;
                            break;
                        case 'f':
                        case 'F':
                            sum = sum * 16 + 15;
                            break;
                    }
                }
            }
            printf("%d\n", sum);
        }
        else if (token_type == FLOAT_CONST)
        {
            printf("%s\n", yytext);
        }
        else if (token_type == ILLEGAL_OCTAL_CONST)
            printf("\033[1;31mError type B at line %d: Illegal octal number \"%s\"\033[0m\n", yylineno, yytext);
        else if (token_type == ILLEGAL_HEX_CONST)
            printf("\033[1;31mError type B at line %d: Illegal hex number \"%s\"\033[0m\n", yylineno, yytext);
    }
}

```

3、使用 flex 生成词法分析器代码 lex.yy.c

Flex 生成的 lex.yy.c 文件包含了词法分析的逻辑，后续使用 GCC 编译并链接生成可执行文件。

```

gaga@gaga-virtual-machine:~/byylsy$ flex lexical.l
gaga@gaga-virtual-machine:~/byylsy$ ll
总计 124
drwxrwxr-x  2 gaga gaga  4096 12月 12 00:59 ./
drwxr-x--- 22 gaga gaga  4096 12月  5 20:42 ../
-rwxrwxr-x  1 gaga gaga 31400 12月 11 23:19 a.out*
-rw-rw-r--  1 gaga gaga  4973 12月 11 23:18 lexical.l
-rw-rw-r--  1 gaga gaga 55644 12月 12 00:59 lex.yy.c
-rw-rw-r--  1 gaga gaga   120 12月 11 20:27 test1.sy
-rw-rw-r--  1 gaga gaga   112 12月 11 20:28 test2.sy
-rw-rw-r--  1 gaga gaga   162 12月 11 20:29 test3.sy
-rw-rw-r--  1 gaga gaga   538 12月 11 21:18 translator.h
-rw-rw-r--  1 gaga gaga  1024 12月 12 00:26 .translator.h.swp
gaga@gaga-virtual-machine:~/byylsy$

```

4、使用 clang 编译器来编译由 flex 生成的词法分析器源代码

lex.yy.c，并链接必要的库以使程序能够正常工作。

lex.yy.c 是 flex 的默认输出文件，它包含了由 flex 根据你的规则自动生成的扫描函数和词法分析器代码。-lfl 告诉编译器在链接阶段包括 libfl 库。libfl 库提供了 flex 的一些功能，特别是用于支持 flex 生成的词法分析器所依赖的函数和数据结构，比如 yytext、yylex()、yylineno 等。确保链接到正确的 lex 库，而这个库包含了 Flex 生成的代码所依赖的一些特定函数和变量。

```
gaga@gaga-virtual-machine:~/byylsy$ clang lex.yy.c -lfl -ll
```

最终生成的 a.out 可执行文件将执行词法分析任务，识别并输出代码中的 Token。

四、测试结果

测试样例 1 未定义字符

```

GNU nano 6.2
int main()
{
    int hjy=22050201;
    int lrh=22050229;
    int wjq=22050106;
    int twx=22050239;
    int i = 1;
    int j = ~1;
    return 0;
}

```

结果如下，可以正确识别错误单元“~”。

```
gaga@gaga-virtual-machine:~/byylsy$ cat test1.sy|./a.out
TYPE      int
ID         main
LPARENT    (
RPARENT    )
LBRACE     {
TYPE      int
ID         hjy
OPERATOR   =
DEC_CONST  22050201
SEMICN     ;
TYPE      int
ID         lrh
OPERATOR   =
DEC_CONST  22050229
SEMICN     ;
TYPE      int
ID         wjq
OPERATOR   =
DEC_CONST  22050106
SEMICN     ;
TYPE      int
ID         twx
OPERATOR   =
DEC_CONST  22050239
SEMICN     ;
TYPE      int
ID         i
OPERATOR   =
DEC_CONST  1
SEMICN     ;
TYPE      int
ID         j
OPERATOR   =
Error type A at Line 8: Invalid character "~"
DEC_CONST  1
SEMICN     ;
KEYWORD    return
OCTAL_CONST 0
SEMICN     ;
RBRACE     }
gaga@gaga-virtual-machine:~/byylsy$
```

测试样例 2 正确代码

```
GNU nano 6.2
int inc()
{
    int lrh=22050229;
    int twx=22050239;
    int wjq=22050106;
    int hjy;
    hjy = 22050201;
    hjy = hjy+1;
}
```

结果如下：

```
gaga@gaga-virtual-machine:~/byylsy$ cat test2.sy|./a.out
TYPE      int
ID         inc
LPARENT    (
RPARENT    )
LBRACE     {
TYPE      int
ID         lrh
OPERATOR   =
DEC_CONST  22050229
SEMICN     ;
TYPE      int
ID         twx
OPERATOR   =
DEC_CONST  22050239
SEMICN     ;
TYPE      int
ID         wjq
OPERATOR   =
DEC_CONST  22050106
SEMICN     ;
TYPE      int
ID         hjy
SEMICN     ;
ID         hjy
OPERATOR   =
DEC_CONST  22050201
SEMICN     ;
ID         hjy
OPERATOR   =
ID         hjy
OPERATOR   +
DEC_CONST  1
SEMICN     ;
RBRACE     }
```

测试样例 3 八进制和十六进制错误

```
GNU nano 6.2
int main()
{
    int hjy=22050201;
    int lrh=22050229;
    int twx=22050239;
    int wjq=22050106;
    int i= 0123;
    int j= 0x3F;
    int s=09;
    int 0x3G;
}
```

结果如下：

```
gaga@gaga-virtual-machine:~/byylsy$ cat test3.sy|./a.out
```

```
TYPE      int
ID        main
LPARENT   (
RPARENT   )
LBRACE    {
TYPE      int
ID        hjy
OPERATOR  =
DEC_CONST 22050201
SEMICN    ;
TYPE      int
ID        lrh
OPERATOR  =
DEC_CONST 22050229
SEMICN    ;
TYPE      int
ID        twx
OPERATOR  =
DEC_CONST 22050239
SEMICN    ;
TYPE      int
ID        wjq
OPERATOR  =
DEC_CONST 22050106
SEMICN    ;
TYPE      int
ID        i
OPERATOR  =
OCTAL_CONST 83
SEMICN    ;
TYPE      int
ID        j
OPERATOR  =
HEX_CONST 63
SEMICN    ;
TYPE      int
ID        s
OPERATOR  =
Error type B at line 9: Illegal octal number '09'
SEMICN    ;
TYPE      int
Error type B at line 10: Illegal hex number '0x3G'
SEMICN    ;
```

```

Error type B at line 9: Illegal octal number '09'
SEMICN      ;
TYPE        int
Error type B at line 10: Illegal hex number '0x3G'
SEMICN      ;
RBRACE      }
gaga@gaga-virtual-machine:~/byylsy$

```

能分别识别出八进制数和十六进制数的错误，并打印相应的错误信息

五、源代码

lexical.l 文件

```
%option yylineno
```

```
%{
```

```
    #include "translator.h"
```

```
%}
```

```
TYPE          int|void|float
```

```
KEYWORD       if|else|while|break|continue|return
```

```
OPERATOR      "+"|"-"|"!"|"*"|"%"|"|"|"/"|"="
```

```
COMPARISON    "=="|"!="|">"|<"|>="|<="
```

```
NONZERO       [1-9]
```

```
DIGIT         [0-9]
```

```
LETTER        [A-Za-z]
```

```
OCTAL_DIGIT   [0-7]
```

```
OCTAL_CONST   0{OCTAL_DIGIT}*
```

```
ILLEGAL_OCTAL_CONST 0[0-9a-wy-zA-WY-Z]({LETTER}|{DIGIT})*
```

```
HEX_PREFIX    0x|0X
```

```
HEX_DIGIT     [0-9a-fA-F]
```

```
HEX_CONST     {HEX_PREFIX}{HEX_DIGIT}+
```

```
ILLEGAL_HEX_CONST {HEX_PREFIX}({LETTER}|{DIGIT})*
```

```
NONDIGIT      {LETTER}|"_"
```

```
ID            {NONDIGIT}({DIGIT}|{NONDIGIT})*
```

```
DEC_CONST     {NONZERO}{DIGIT}*
```

```
FLOAT_CONST   [0-9]+ "." [0-9]+ ([eE][+-]? [0-9]+)?
```

```
COMMENT1      "/*" [^]* "*" + ([^/] [^]* "*" +)* "/"
```

```
COMMENT2      "/*".*
```

```
%%
```

```
{TYPE}        { printf("\033[1;32mTYPE\033[0m\t\t%s\n",yytext);
```

```

return TYPE; }
{OCTAL_CONST}
{ printf("\033[1;32mOCTAL_CONST\033[0m\t"); return OCTAL_CONST; }
{ILLEGAL_OCTAL_CONST}    { return ILLEGAL_OCTAL_CONST; }
{HEX_CONST}              { printf("\033[1;32mHEX_CONST\033[0m\t");
return HEX_CONST; }
{ILLEGAL_HEX_CONST}      { return ILLEGAL_HEX_CONST; }
{DEC_CONST}
{      printf("\033[1;32mDEC_CONST\033[0m\t%s\n",yytext);      return
DEC_CONST; }
{FLOAT_CONST}
{      printf("\033[1;32mFLOAT_CONST\033[0m\t%s\n",yytext);      return
FLOAT_CONST; }
{KEYWORD}
{      printf("\033[1;32mKEYWORD\033[0m\t\t%s\n",yytext);      return
KEYWORD; }
{ID}                      { printf("\033[1;32mID\033[0m\t\t%s\n",yytext);
return ID; }
{OPERATOR}
{ printf("\033[1;32mOPERATOR\033[0m\t\t%s\n",yytext); return OPERATOR; }
{COMPARISON}
{      printf("\033[1;32mCOMPARISON\033[0m\t\t%s\n",yytext);      return
COMPARISON; }
"("
{ printf("\033[1;32mLPARENT\033[0m\t\t%s\n",yytext); return LPARENT; }
")"
{ printf("\033[1;32mRPARENT\033[0m\t\t%s\n",yytext); return RPARENT; }
"["
{      printf("\033[1;32mLBRACKET\033[0m\t\t%s\n",yytext);      return
LBRACKET; }
"]"
{      printf("\033[1;32mRBRACKET\033[0m\t\t%s\n",yytext);      return
RBRACKET; }
"{"
{ printf("\033[1;32mLBRACE\033[0m\t\t%s\n",yytext); return LBRACE; }
"}"
{ printf("\033[1;32mRBRACE\033[0m\t\t%s\n",yytext); return RBRACE; }
";"
{ printf("\033[1;32mSEMICN\033[0m\t\t%s\n",yytext); return SEMICN; }
","
{ printf("\033[1;32mCOMMA\033[0m\t\t%s\n",yytext); return COMMA; }
"&&"          { printf("\033[1;32mAND\033[0m\t\t%s\n",yytext);
return AND; }
"||"          { printf("\033[1;32mOR\033[0m\t\t%s\n",yytext);

```

```

return OR; }
{COMMENT1}{COMMENT2}    {}
[ \t\n]                  {}
.                          { return UNEXPECTED; }

%%

int main()
{
    int token_type;
    while (token_type = yylex())
    {
        if (token_type == UNEXPECTED)
            printf("\033[1;31mError type A at Line %d: Invalid character\n", yylineno, yytext);
        else if (token_type == OCTAL_CONST)
        {
            int sum = 0;
            for (int i = 1; i < yyleng; ++i)
                sum = sum * 8 + (yytext[i] - '0');
            printf("%d\n", sum);
        }
        else if (token_type == HEX_CONST)
        {
            int sum = 0;
            for (int i = 2; i < yyleng; ++i)
            {
                if (yytext[i] <= '9' && yytext[i] >= '0')
                    sum = sum * 16 + (yytext[i] - '0');
                else
                {
                    switch (yytext[i])
                    {
                        case 'a':
                        case 'A':
                            sum = sum * 16 + 10;
                            break;
                        case 'b':
                        case 'B':
                            sum = sum * 16 + 11;
                            break;
                        case 'c':
                        case 'C':
                            sum = sum * 16 + 12;

```

```

        break;
    case 'd':
    case 'D':
        sum = sum * 16 + 13;
        break;
    case 'e':
    case 'E':
        sum = sum * 16 + 14;
        break;
    case 'f':
    case 'F':
        sum = sum * 16 + 15;
        break;
    }
}
}
printf("%d\n", sum);
}
else if (token_type == FLOAT_CONST)
{
    printf("%s\n", yytext);
}
else if (token_type == ILLEGAL_OCTAL_CONST)
    printf("\033[1;31mError type B at line %d: Illegal octal number\n", yylineno, yytext);
else if (token_type == ILLEGAL_HEX_CONST)
    printf("\033[1;31mError type B at line %d: Illegal hex number\n", yylineno, yytext);
}
}
}

```

Translator.h

//对需要识别的词法类型进行了 enum 枚举

```
#include <stdio.h>
```

```
enum target
```

```

{
    TYPE = 1,
    KEYWORD,
    OPERATOR,
    COMPARISON,
    NONZERO,
    DIGIT,
    LETTER,
    OCTAL_DIGIT,

```

```
OCTAL_CONST,  
ILLEGAL_OCTAL_CONST,  
HEX_PREFIX,  
HEX_DIGIT,  
HEX_CONST,  
ILLEGAL_HEX_CONST,  
NONDIGIT,  
ID,  
DEC_CONST,  
FLOAT_CONST,  
COMMENT1,  
COMMENT2,  
UNEXPECTED,  
LPARENT,  
RPARENT,  
LBRACKET,  
RBRACKET,  
LBRACE,  
RBRACE,  
SEMICN,  
COMMA,  
AND,  
OR  
};
```